Martin-Luther-Universität Halle-Wittenberg
Faculty of Natural Science II
Institute of Physics

Physics Modelling Assistant

Using Agentic Artificial Intelligence in physics modelling

Bachelor's Thesis
for the Award of the Academic Degree
Bachelor of Science
in Physik und Digitale Technologien

submitted by: **Janis Felix Jacobi**
Supervisor 1: Prof. Dr. Niels Schröter
Supervisor 2: Prof. Dr. Samir Lounis

submission: Halle (Saale), 23rd August 2026

Contents

Figure Index

Table Index

Glossary

Agent a singular AI system which is commonly part of an Agentic AI system 3, 9–13

Agentic AI AI system with enhanced capabilities (subsubsection 2.1.3) 2, 3, 9–11

AI Artificial Intelligence 2–12

API key application programming interface key; a unique and secret identifier granting access to a service 3, 12

context-window a size describing how far back previous output token are considered in the LLM 3, 6

LLM Large Language Model 2, 3, 5–13

logit an LLMs output for every token indicating how well it is suited to be the next token. Larger values indicate a better fit. [26, p. 4] 3, 6, 7

token input for a LLMs transformation function 3, 5–9, 12

1 Introduction

Artificial Intelligence can be of great assistance in scientific research [41]. As shown by Chugunova et al. [5], researchers in natural science are among the groups most familiar with AI among scientists. There is a wide range of application stretching from finding sources in the literature to aiding in the presenting research findings and including the steps in between. It can help in forming hypothesis and in evaluating them. This includes the design and control of practical experiments as well as aiding during simulations especially with the program implementation.[41]

To improve the work with AI in scientific research automated AI research pipelines where developed in the last two years. The first fully autonomous approach was published by Lu et al. in 2024[32]. Their framework called "The AI Scientist"[32] identifies interesting research questions, plans and executes according experiments, interprets the results and writes a paper stating its findings in a multistep process. Similar approaches where made by Team et al. with the "InternAgent"[38], which provides an interface for human feedback into the system. This is realised by the "Orchestration Agent"[38], which manages how and when human input is introduced into the system which itself consists of the four distinct tasks of "Survey", "Code Review", "Assessment" and "Idea Innovation"[38]. The "Orchestration Agent"[38] manages how the agents responsible for the interactions between the four tasks and how the findings are presented to the human. This system shows some resemblance to pure human research group with individuals specialized in specific parts of the work while a supervisor coordinates the process. The InternAgent was able to outperform the "AI-Scientist-V2" [43], a successor of "The AI Scientist"[32], as shown in the tests conducted by the InternAgent Team [38].

The publication of the InternAgent [38] also introduced measuring the costs of the AI usage during the process. The "InternAgent" [38] outperformed both instances of the "AI-Scientist" [32][43] at less than $\frac{1}{3}$ of the cost.

A less automated approach was made by Shao et al. with the introduction of "SciSciGPT" [36]. This system is for Science of Science research, which is why SciSciGPT is equipped with data management component amongst a literature review and an analytics component. Similar to the "InternAgent" [38] [36] uses a component to handle the coordination of subtasks, which is called "ResearchManager" [38] who gets the results of every subtask along with an evaluation from the "EvaluationSpecialist" [36], indicating if a rerun is needed. In contrast to the previously discussed frameworks "SciSciGPT" [36] does not generate its own research ideas, instead the human researcher makes a request (a text prompt) and the system takes this as its input. As mentioned by the authors Shao et al., "SciSciGPT" [36] is useful for more methodical tasks thereby allowing the human researcher to focus on the interpretation of the results.

As of 1st June 2026 the latest advancement in the field was made by Louapre, Niklaus, and Tunstall by presenting the "physics-intern"[31], "a multi-agent scaffolding system that takes a physics problem and works through it autonomously"[31]. Their approach is similar to previous systems, using an orchestrator to manage a research and computation component. The results are reviewed and checked against the literature before the critique is passed to the "Strategy Planner"[31]. The "Strategy Planner"[31] has access to a literature "Background Survey"[31] and can adjust the orchestration strategy.

This Thesis aims to develop a system with functionality similar to the "physics-intern"[31] (at the time the work on this Thesis began, the "physics-intern"[31] was not yet published). The Physics Modelling Assistant, which is to be developed, should take a physics problem/modelling request and craft a model and its implementation to answer the request. Once this system is running, the influence of the systems configuration on its performance will be explored. With this work I want to contribute to a better understanding of AI and a more efficient use of it in scientific discovery.

2 Theoretical Foundations

Before going in to the details of AI based systems some definitions of important concepts will be made.

2.1 Definitions

2.1.1 Artificial Intelligence

Artificial Intelligence (AI) describes the functionality of a system. There is no universally accepted definition for AI. For the purpose of this Thesis, the definition from Russell and Norvig [35, p. 22-26] will be used: They describe AI as being a rational actor (rational agent), which acts in a way that achieves the best result based on the expectation of what the result should be ([35, p, 26], translated). This definition allows for a variety of systems to be considered an AI and accounts for the uncertainty regarding the goal the system should work towards.

2.1.2 Large Language Model

A Language Model is a "Transformer Language Model"[44] as described by Zhao et al. Then a multistep process is used to obtain an output:

1. The input text is divided into tokens, these are small words or parts of words. [44]
2. The input tokens are plugged into a transformation function, which predicts the most likely next token based on its parameters, these include the previously generated tokens. The number of previous tokens considered is called context window. [28]

This results in a set of output tokens which are a human readable text and provide the most likely output, which ideally is the result desired by the one providing the input. The process of developing a model, which can do the described procedure well enough is a highly complex task and beyond the scope of this Thesis.

The distinction between Language Models and Large Language Models is in the number of internal parameters a model uses. Internal parameters include input and output tokens as well as the parameters discussed in subsubsection 2.2.3, subsubsection 2.2.4 and subsubsection 2.2.5. As stated in *A Survey of Large Language Models*[44], there is no hard cut-off for a Language Model being Large, but commonly models have around $10 \cdot 10^9$ parameters when considered Large Language Models (LLMs).

A LLM is one way of implementing AI.

2.1.3 Agentic AI

As described by Bandi et al., Agentic AIs are "autonomous, goal-driven systems that can operate on their own for long periods, requiring minimal human supervision"[3, section:3. Results]. In order to achieve this behaviour, these systems combine abilities that are beyond singular LLMs. Bandi et al. highlight the "strategic planning" [3, section:3. Results] capabilities, the preservation of the tasks context over time [3], the usage of "external tools to extend [the systems] capabilities" [3] and the collaboration of individual Agents (this means AI-systems) with other Agents[3].

2.2 Decoding the output of LLMs

In subsubsection 2.1.2 LLM is defined based on the functionality it provides. In order to understand how the behaviour of LLMs can be modified, a more formal definition will be given, based on the work of Ji et al. in *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers* [26].

2.2.1 LLM as a function which assigns probabilities

A LLM can be described as a function f with internal parameters θ. Additionally, f takes a set of input token $t_{in} = (i_1, ..., i_N)$ where $N \in \mathbb{N}$ is the number of input tokens, and a set of previous output token $v_{out,prev} = (o_{-1}, ..., o_{-M})$ where $M \in \mathbb{N}$ is the number of previous output tokens considered, which is called the context-window with size M. The function $f(\theta, t_{in}, t_{outp,prev})$ assigns a probability $s(v)$ to every token $v \in \mathcal{V}$ where $\mathcal{V}$ is the vocabulary of tokens known to the LLM:

$$f(\theta, t_{in}, t_{outp,prev}) = \begin{pmatrix} s(v'_1) \\ ... \\ s(v'_Z) \end{pmatrix} = s(v) \quad \text{where } Z \text{ is the size of the vocabulary } \mathcal{V} \tag{2.2.1}$$

v'_i with $i \in [1, Z] \subset \mathbb{N}$ is an element from the ordered set of $\mathcal{V}'$, which allows to map the probabilities to the . The probability indicates how well a token is suited to be the next output token[26, p. 4]. The probabilities $s(v)$ are called "logits"[26, p. 4] and not normalized by default, larger values of a logit signal a better fit for the corresponding token to be the next token.

The LLM outputs a probability distribution including every token from its vocabulary. To choose a token based on this probability distribution, Ji et al. state that the following optimization problem needs to be solved and labelled "MASTER PROBLEM"[26, p. 5]:

$$q^* = \arg\max_{q \in \Delta(\mathcal{V})} \left[\underbrace{\langle q, s \rangle}_{\sum_{v \in \mathcal{V}} q(v) \cdot s(v)} - \lambda \Omega(q) \right] \tag{2.2.2}$$

The used properties are:

- q^* is the probability distribution maximizing Equation 2.2.2
- $\Delta(\mathcal{V})$ is the set of all possible probability distributions
- q is a single probability distribution distributions for a given vocabulary $\mathcal{V}$
- s is the initial vector of logits the LLM produces
- $\Omega(q)$ is a regularisation function, which can push q^* towards a certain distribution
- λ is the strength of the regularisation function and $\lambda \geq 0$

There are different approaches for finding q^* and eventually getting a single token from it, the once relevant to this Thesis will be discussed in the following.

2.2.2 Greedy Decoding [26, p. 10]

This method uses no special regularisation function $\Omega(q)$, respectively sets $\lambda = 0$. This simplifies Equation 2.2.2 to $q^* = \arg\max_{q\in\Delta(\mathcal{V})}\langle q, s\rangle$. The resulting q^* vector has the value 1 in the line where s has its maximum, while every other line of q^* is 0. This means, that the next token will always be the most likely token according to the LLMs calculations. If multiple token have the highest probability $s(v)$, a single token is picked.[26, p. 10]

2.2.3 Softmax Decoding

For Softmax Decoding, the regularisation function is chosen as $\Omega(q) = -\sum_{v\in\mathcal{V}} q(v)\log(q(v))$, which is the negative entropy of q. Using this in solving Equation 2.2.2 leads to the following optimal distribution q^*, where $q^*(v)$ os the probability per token[26, p. 10-12]:

$$q^*(v) = \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u\in\mathcal{V}} e^{\frac{s(u)}{\lambda}}} \tag{2.2.3}$$

This distribution is normalised and has the parameter λ, which controls how the distribution q^* is spread. For larger values of λ, the probability is spread more evenly across all tokens. This results in tokens the LLMs predicts to be less likely being more probable to be the next tokens. When λ becomes very small, the token the LLM calculates to me suitable become more likely. For the edge case $\lambda = 0$, simply Greedy Decoding is performed as it is discussed in subsubsection 2.2.2. The next token is then selected from the final probability distribution.

The parameter λ is one way of controlling the general randomness of the output. In the context of LLM it is often associated with the creativity of a model and called **Temperature** T [26, p. 10-12].

2.2.4 Top-k Decoding

Top-k Decoding extends Softmax Decoding (subsubsection 2.2.3) by selecting the k ($k \in \mathbb{N} \setminus k = 0$) token with the highest logit values[26, p. 12]. These tokens form a subset of the vocabulary called $\mathcal{V}_k$ on which Softmax Decoding is performed, while the probability for every other token is set to 0. This gives the following probability distribution[26, p. 12]:

$$q^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u\in\mathcal{V}_k} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_k \\ 0 & \text{else} \end{cases} \tag{2.2.4}$$

For $k >= |\mathcal{V}|$, Top-k Decoding is equivalent to the standard Softmax Decoding (subsubsection 2.2.3). Setting $k = 1$ results in a result similar to Greedy Decoding (see subsubsection 2.2.2).

Top-k Decoding provides another parameter called **top_k**, which can be used to shape the output of an AI system [26, p. 12].

2.2.5 Top-p Decoding

Top-p Decoding is another extension to Softmax Decoding (subsubsection 2.2.3) also selecting a subset of token called $\mathcal{V}_p$ from the vocabulary $\mathcal{V}$. In contrast the Top-k Decoding (subsubsection 2.2.4), Top-p Decoding defines a certain amount of probability $p \in (\ 0, 1\ ]$, which will be considered for the Softmax Decoding. How this works is that the token with the highest probability $s(v)$ from the set $\mathcal{V} \setminus \mathcal{V}_p$ will be added to the set $\mathcal{V}_p$. This will be repeated until the sum of the probabilities of all token $v \in \mathcal{V}_p$ exceeds the value of p. Then the probability of the tokens in $\mathcal{V} \setminus \mathcal{V}_p$ will be set to 0, while Softmax Decoding (subsubsection 2.2.3) will be performed on $\mathcal{V}_p$. The resulting probability distribution is (from [26]):

$$q^*(v) = \begin{cases} \frac{e^{\frac{s(v)}{\lambda}}}{\sum_{u \in \mathcal{V}_p} e^{\frac{s(u)}{\lambda}}} & \text{for } v \in \mathcal{V}_p \\ 0 & \text{else} \end{cases} \tag{2.2.5}$$

For $p = 1$ Top-p Decoding is equivalent to Softmax Decoding. For $p \to 0$, only the token with the highest probability gets a non 0 probability, which makes this case equivalent to Greedy Decoding (subsubsection 2.2.2).

Top-p Decoding allows for dynamic adaptation based on how sharp or spread out the probability distribution given by the LLM is. For a sharp distribution, only the view very likely token are given a non 0 probability, whereas for a flatter distribution a higher number of tokens is selected and thus is a possible output token.

The resulting parameter **top_p** is yet another parameter often used in the field of LLM to modify a systems output [26, p. 13].

2.3 Other parameters that can influence an AI systems performance

2.3.1 Selection of the LLM

As shown by numerous benchmarks ([21], [22]) different LLM perform differently.

One of the most noticeable differences in LLM systems is the number of internal parameters used. The general assumption is that more parameters lead to better performance. Another important feature is the amount of data used to train a LLM. As shown by Kaplan et al.([27, p. 5]) combining the number of internal parameters N a model uses and the number of tokens D used in training a model can predict the "cross-entropy loss" [27, p. 6] L of a LLM:

$$L(N, D) = \left[\left(\frac{N_C}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_C}{D} \right]^{\alpha_D} \tag{2.3.1}$$

- $L(N, D)$ is the "cross-entropy loss"[27, p. 6]. Lower Values indicate a better model.
- N_C is the critical number of internal parameter until which Equation 2.3.1 is applicable. $N_C \approx 8.8 \cdot 10^{13}$
- N is the number of internal parameters the model uses
- D_C is the critical number of tokens until which Equation 2.3.1 is applicable. $D_C \approx 5.4 \cdot 10^{13}$
- α_N is the exponent for predicting $L(N)$. $\alpha_N \approx 0.076$
- α_D is the exponent for predicting $L(D)$. $\alpha_D \approx 0.095$

There are also other influences on a models performance, Kaplan et al. discus the influence of the number of training steps or the batch size (tokens per step) [27]. Liu et al. [30] showed the relevance of how the training data are composed, highlighting the increases in predicting a models performance.

2.3.2 Tool use for LLMs

Since all LLMs have a limited amount of training data, the model on its own will not be able to answer some questions correctly. In order to mitigate this LLMs are getting fed extra information to help them answer the question at hand. Realizing this via human input is inefficient and impractical, especially when the human itself is not very knowledgeable in the question at hand. For that reason, LLM systems are getting equipped with tools, allowing them to access information not included in their training data. As shown by omeili, Shuster, and Weston [34] and Thoppilan et al. [40], equipping a LLM with some sort of web search tool allows to reduces the false output of the system. Note that tool use is one of the features making an AI an Agentic AI as discussed in subsubsection 2.1.3.

Another common tool is file reader, they can be able to parse various formats like PDF, json, or plain text. Code execution tools allow a LLM run code it generates. This introduces the problem of untrusted code execution, since the model could generate malicious code accessing private information or modifying important data. Executing untrusted code should therefore be done in a container separate from important data and systems. As stated by Marchand et al. [33], these containers called Sandboxes are not always fully secure strong LLM models are able to escape them.

2.4 Orchestrating AIs to behave as an Agentic AI

As mentioned in subsubsection 2.1.3, combining multiple AI systems into one enables the resulting system to perform more complex tasks and achieve better results. The individual AI systems of an Agentic AI system are commonly referred to as Agents. In the following the common approaches on how Agents can work together will be presented.

2.4.1 Orchestration Architectures

The Sequential Pipeline [29, p. 3] is a linear chain where each Agent is executed one after another [29]. Individual Agents or their tools can be rerun, as long as the pipeline is still in the corresponding step, but once an Agent is done, it can not be called again (in the same run). This results in a very deterministic behaviour, which makes the design process relatively simple and allows to locate errors easily [29].

The Parallel Fan-Out with Merge [29, p. 4] differs from the Sequential Pipeline (paragraph 2.4.1) by allowing certain Agents to run simultaneously. After the parallel Agents are done with their tasks the results are merged by another Agent. By introducing an Agent before the parallelization each Agent running in parallel will only get the information necessary to its task therefore having to deal with less noise. The Agent responsible for merging will handle conflict like contradicting information. This parallel design allows for the highest throughput compared to the other architectures [29, p. 7].

The Hierarchical Supervisor-Worker [29, p. 4-5] architecture introduces a supervisor Agent that dynamically decides which Agents (Subagents) will be called and which tasks they should handle. Furthermore, the supervisor Agent reviews the Subagent results and will run the tasks again if the confidence in its results being correct is low, thereby forming a feedback loop. this design allows the Agentic AI system to dynamically assign more difficult tasks to more powerful Agents. This hierarchical architecture produces better results than the Sequential Pipeline (paragraph 2.4.1) and the parallel design (paragraph 2.4.1) but is also more costly regarding the execution time [29, p. 7].

The Reflexive Self-Correcting Loop [29, p. 5] executes the Agents in a predefined order similar to the Sequential Pipeline (paragraph 2.4.1) but adds a verification step. This will be performed by an Agent, which will either deem the result as correct (enough) and make it the systems output or send it back for correction. Before the result is introduced back into the system it will be critiqued, thereby stating the problems encountered and allowing for specific correction. This architecture allows refine the output over time instead of simply rerunning the entire system. The Reflexive Self-Correction Loop [29, p. 5] achieves the highest accuracy but also comes at the highest cost of all architectures discussed [29, p. 7].

To interpret the performances of the architectures it should be mentioned that Kulkarni and Kulkarni [29, p. 14-15] focused on the extraction capabilities from financial documents. For that reason the performance could vary for different tasks and fields. For designing Agentic AI systems the combination of different architectures is also possible.

2.4.2 Communication inside Agentic AI systems

Communication protocols are used to define how the different components of an Agentic AI system. An orchestration framework can implement these protocols or some of them. To get from the text an LLM puts out to calling a tool or communicating with another Agent a layered architecture is used, which identifies when and where to route an LLMs output to a tool or another LLM [17].

The Model Context Protocol (MCP) [14] (documentation: [39]) handles how the underlying LLM of an Agent uses tools.

The Agent Communication Protocol (ACP) [14] (documentation: [12]) allows different Agents to communicate with each other. It is based on a client-server architecture and uses “RESTful” communication [18].

The Agent-to-Agent Protocol (A2A) [14] (documentation: [37]) handles Agent interaction to allow effective cooperation. This is achieved by Agents communicating their capabilities [18].

The Agent Network Protocol (ANP) [14] (documentation: [4]) enables decentralised communication between Agents. It allows for secure interactions over the internet [18].

2.4.3 Memory

In the context of Agentic AI memory is an Agents awareness of previous actions. Short-term memory is the context for the task the Agent currently performs. Long-term memory allows an Agent to access information obtained from previous runs [14]. Other kinds of memory include “semantic memory” [14], “procedural memory” [14] and “episodic memory” [14] but will not be discussed in this Thesis.

2.5 Evaluation of AI systems

In order to measure the configurations influence on an AI system, be it a LLM or an Agentic AI, a metric for the systems performance is needed. One approach is giving the AI system one multiple choice question at a time and deem each answer either right or wrong [20]. The total score over the set of questions determines the system performance. This approach is used by [22] [19] for their [24].

Another option are questions without answers to choose from which is used by “SciBench”[42]. The answers are usually short and need to be matched exactly as they are compared against the expected answer. The performance over all questions is accumulated and gives a final evaluation of the tested system.

In order to properly assess how well AI systems handle more complex tasks human grading can be used. This is use by Arena Intelligence [1], where users write a prompt and then decide from two anonymous answers which they like best. Human grading is also used by in “SciEx” [15], where experts evaluate the results. As acknowledged by Dinh et al. human grading is not feasible for large scale benchmarking.

2.5.1 CritPt

A first step to bridge the gap between simple answer comparison and human level evaluation is done by Zhu et al. with the introduction of “CritPt” [45]. In the first step the system, which is to be evaluated, is prompted with the question and generates its answer. Secondly, Zhu et al. [45] guides the system to implement the needed for the automatic evaluation.

There are three types of questions requiring either a numerical value, a symbolic expression or python code, which can be executed. Composite answers are only graded to be correct if all individual parts are correct [45].

3 Design of the Physics Modelling Assistant

In order to easily enable future work on and with the Physics Modelling Assistant the choice was made to only use services, which are open source and free of charge.

3.1 Orchestration Framework: CrewAI

The decision was made not to design a new framework for orchestrating AIs to save time on this software development heavy task and instead focus on the Physic Modelling Assistants configurations. Derouiche, Brahmi, and Mazeni [14] discuss a number of frameworks varying in complexity and beginner friendliness. For this Thesis CrewAI [10] will be used for its role focused style [14] consisting of Agents and Tasks [10]. Each Agent can be configured individually, which includes using different LLMs for different tasks. Every task has its own description and specification including the Agent assigned to it. On execution of a task, its definition will be merged with the assigned Agent which provides the input for the LLM used. An Agents behaviour will be defined by its role, goal and backstory. The role a very short description comparable to a job title, the goal describes what the Agent does while working similar to a job description while the backstory provides details on the agents behaviour like preferences or advice on how to approach tasks. CrewAI allows to access any LLM provided the user has a valid API key. This LLM can be assigned parameters including the discussed Temperature (subsubsection 2.2.3) and top_p (subsubsection 2.2.5), the number of tokens to be used and others. The Task gets a description and specifications regarding the output and output format along with the designated agent [11]. Tools can be assigned to both Agents and Tasks with CrewAI providing ready to use tools [6] but also allowing to add custom tools.

The organisation of multiple CrewAI Agents and tasks is managed by the so called crew. It is responsible for validating the configurations beforehand and then execute the system. As stated by Derouiche, Brahmi, and Mazeni [14] CrewAI implements an orchestration combining elements from the ACP, A2A and ANP (see subsubsection 2.4.2). The CrewAI documentation [10] and the source code available on GitHub [9] show a sequential and a hierarchical execution approach. The sequential execution will run the tasks in their order of definition they are defined inside the code which gives a Sequential Pipeline (see paragraph 2.4.1). This can be modified into a Parallel Fan-Out with Merge (see paragraph 2.4.1) by allowing certain tasks to execute asynchronously [8], the merging is handles by specifying a task, which should wait with its execution until the parallel tasks are done. The hierarchical execution style of CrewAI enables a Hierarchical Supervisor-Worker style by delegating tasks or aspects of them. Implementing a Reflexive Self-Correcting Loop (see paragraph 2.4.1) with CrewAI is possible by making use of Flows [7], which allow for conditional calls and the coupling of multiple crews.

In order to achieve a deterministic workflow the Sequential Pipeline architecture (paragraph 2.4.1) is used.

3.2 LLMs used

In order to use LLMs effectively in an automated pipeline a provider offering a program access to its LLM is needed. For this reason the “Scalable AI Accellarator (SAIA)” [16] provided by the “Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen” (GWDG) [25]. The LLMs offered by SAIA can be accessed via an API key obtained from the GWDG. The available LLMs are listed at the “SAIA” webpage [13] but also via a direct request to the server hosting the models. To identify the most capable of the models offered the benchmarking provided by Artificial Analysis [2] with the LLMs being evaluated on the following benchmarks: AA-LCR (long context reasoning), GPQA Diamond (scientific reasoning), SciCode (science and coding) and CritPt (subsubsection 2.5.1). The result seen in Table 1 conclude qwen3.6 27B (27 billion parameters) to be the best model for long context reasoning, while qwen3.5 397B A17B (397 billion parameters in total, 17 billion active at a time) performs best in scientific reasoning. glm-4.7 (358 billion parameters [23]) achieves the best result in science and coding. For the CritPt benchmark both glm-4.7 and qwen3.5 397B A17B achieve the best score among the available models, with both of them scoring at only

1.7 %, indicating the difficulty of this benchmark.

Table 1: Benchmarking results for the LLMs provided by SAIA [16]. The benchmarking results are taken from Artificial Analysis [2]. The chosen benchmarks are: AA-LCR evaluates long context reasoning, GPQA Diamnond evaluates scientific reasoning, SciCode evaluates coding and science, CritPt evaluates physics reasoning

model	AA-LCR in %	GPQA Diamond in %	SciCode in %	CritPt in %
apertus 70B Instruct	0	27	6	0
deepSeek V4 Flash	33	72	37	0
devstral 2	30	53	29	0
gemma 4 31B	62	86	43	1
glm-4.7	64	86	45	1.7
llama 3.1 8B	16	26	13	0
mistral Medium 3.5	61	75	40	0
gpt-oss-120B (high)	51	78	39	1
qwen3 30B A3B 2507	59	71	33	0
qwen3 coder next	40	74	32	0
qwen3 omni 30B A3B	0	73	31	0
qwen3.5 122B A10B	67	86	42	1
qwen3.5 397B A17B	66	89	42	1.7
qwen3.6 27B	69	84	40	1
qwen3.6 35B A3B	64	84	36	0

3.3 Tools used

To enhance the Physics modelling Assistants capabilities certain Agents are equipped with Tools

4 Benchmarking

Figure 1: testC

References

[1] Arena Intelligence 2026, ed. *Arena*. URL: https://arena.ai/how-it-works (visited on 07/29/2026) (cit. on p. 11).

[2] Artificial Analysis, ed. *Independent analysis of AI*. URL: https://artificialanalysis.ai/ (visited on 07/30/2026) (cit. on pp. 12, 13).

[3] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. “The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges”. In: *Future Internet* 17.9 (2025). ISSN: 1999-5903. DOI: 10.3390/fi17090404. URL: https://www.mdpi.com/1999-5903/17/9/404 (cit. on pp. 5, 6).

[4] Gaowei Chang. *Agent Network Protocol*. URL: https://agent-network-protocol.com/ (visited on 07/29/2026) (cit. on p. 10).

[5] Marina Chugunova, Dietmar Harhoff, Katharina Hölzle, Verena Kaschub, Sonal Malagimani, Ulrike Morgalla, and Robert Rose. “Who uses AI in research, and for what? Large-scale survey evidence from Germany”. In: *Research Policy* 55.2 (2026), p. 105381. ISSN: 0048-7333. DOI: https://doi.org/10.1016/j.respol.2025.105381. URL: https://www.sciencedirect.com/science/article/pii/S0048733325002100 (cit. on p. 4).

[6] CrewAI. *crewai tools*. URL: https://github.com/crewAIInc/crewAI/tree/main/lib/crewai-tools/src/crewai_tools (visited on 07/30/2026) (cit. on p. 12).

[7] CrewAI. *Flows*. URL: https://docs.crewai.com/v1.15.9/en/concepts/flows (visited on 07/30/2026) (cit. on p. 12).

[8] CrewAI. *Tasks*. URL: https://docs.crewai.com/v1.15.9/en/concepts/tasks#asynchronous-execution (visited on 07/30/2026) (cit. on p. 12).

[9] crewai. *crew.py*. 04/22/2026. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/crew.py (visited on 04/22/2026) (cit. on p. 12).

[10] crewai. *crewAi*. 04/22/2026. URL: https://crewai.com/ (visited on 04/22/2026) (cit. on p. 12).

[11] crewai. *task.py*. 04/22/2026. URL: https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/task.py (visited on 04/22/2026) (cit. on p. 12).

[12] Linux Foundation AI & Data. *Agent Communication Protocol*. URL: https://agentcommunicationprotocol.dev/about/mission-and-team (visited on 07/29/2026) (cit. on p. 10).

[13] “SAIA”. In: (). Ed. by Gesellschaft für wissenschafliche Datenverarbeitung mbH Göttingen. URL: https://docs.hpc.gwdg.de/services/ai-services/saia/index.html#api-request (visited on 07/30/2026) (cit. on p. 12).

[14] Hana Derouiche, Zaki Brahmi, and Haithem Mazeni. *Agentic AI Frameworks: Architectures, Protocols, and Design Challenges*. 2025. arXiv: 2508.10146 [cs.AI]. URL: https://arxiv.org/abs/2508.10146 (visited on 07/28/2026) (cit. on pp. 10–12).

[15] Tu Anh Dinh, Carlos Mullov, Leonard Bärmann, et al. *SciEx: Benchmarking Large Language Models on Scientific Exams with Human Expert Grading and Automatic Grading*. 2024. arXiv: 2406.10421 [cs.CL]. URL: https://arxiv.org/abs/2406.10421 (cit. on p. 11).

[16] Ali Doosthosseini, Jonathan Decker, Hendrik Nolte, and Julian Kunkel. “SAIA: a seamless Slurm-native solution for HPC-based services”. In: *The Journal of Supercomputing* 82.7 (05/2026), p. 403. ISSN:

1573-0484. DOI: 10.1007/s11227-026-08508-3. URL: https://doi.org/10.1007/s11227-026-08508-3 (cit. on pp. 12, 13).

[17] Yogesh K. Dwivedi, Mohamed Y. I. Helal, Ibrahim A. Elgendy, Rasha Alahmad, Paul Walton, Ayoung Suh, Vinay Singh, and Il Jeon. “Agentic AI Systems: What It Is and Isn’t”. In: *Global Business and Organizational Excellence* 45.3 (2026), pp. 253–263. DOI: https://doi.org/10.1002/joe.70018. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/joe.70018. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/joe.70018 (visited on 07/28/2026) (cit. on p. 10).

[18] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. 2025. arXiv: 2505.02279 [cs.AI]. URL: https://arxiv.org/abs/2505.02279 (visited on 07/29/2026) (cit. on p. 10).

[19] Clémentine Fourrier and Mishig. *What’s going on with the Open LLM Leaderboard?* URL: https://github.com/huggingface/blog/blob/main/open-llm-leaderboard-mmlu.md (visited on 07/29/2026) (cit. on p. 11).

[20] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. *Measuring Massive Multitask Language Understanding*. 2021. arXiv: 2009.03300 [cs.CY]. URL: https://arxiv.org/abs/2009.03300 (visited on 07/29/2026) (cit. on p. 11).

[21] HuggingFace. *Agent Arena*. URL: https://huggingface.co/spaces/lmarena-ai/arena-leaderboard (visited on 07/28/2026) (cit. on p. 8).

[22] HuggingFace. *Artificial Analysis LLM Leaderboard*. URL: https://huggingface.co/spaces/ArtificialAnalysis/LLM-Performance-Leaderboard (visited on 07/28/2026) (cit. on pp. 8, 11).

[23] HuggingFace. *GLM-4.7-FP8*. URL: https://huggingface.co/zai-org/GLM-4.7-FP8 (visited on 07/30/2026) (cit. on p. 12).

[24] HuggingFace. *Open LLM Leaderboard Archived*. URL: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard#/?search=llama (visited on 07/29/2026) (cit. on p. 11).

[25] *Imprint*. URL: https://gwdg.de/imprint/ (visited on 07/30/2026) (cit. on p. 12).

[26] Xiaotong Ji, Rasul Tutunov, Matthieu Zimmer, and Haitham Bou-Ammar. *Decoding as Optimisation on the Probability Simplex: From Top-K to Top-P (Nucleus) to Best-of-K Samplers*. 2026. arXiv: 2602.18292 [cs.LG]. URL: https://arxiv.org/abs/2602.18292 (visited on 07/27/2026) (cit. on pp. 3, 6–8).

[27] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG]. URL: https://arxiv.org/abs/2001.08361 (visited on 07/28/2026) (cit. on pp. 8, 9).

[28] Vikram Krishnamurthy. *LLMs as High-Dimensional Nonlinear Autoregressive Models with Attention: Training, Alignment and Inference*. 2026. arXiv: 2602.00426 [cs.LG]. URL: https://arxiv.org/abs/2602.00426 (cit. on p. 5).

[29] Siddhant Kulkarni and Yukta Kulkarni. *Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and*

Production Scaling Strategies. 2026. arXiv: 2603.22651 [cs.AI]. URL: https://arxiv.org/abs/2603.22651 (visited on 07/28/2026) (cit. on pp. 9, 10).

[30] Emmy Liu, Amanda Bertsch, Lintang Sutawika, et al. *Not-Just-Scaling Laws: Towards a Better Understanding of the Downstream Impact of Language Model Design Decisions*. 2026. arXiv: 2503.03862 [cs.CL]. URL: https://arxiv.org/abs/2503.03862 (visited on 07/28/2026) (cit. on p. 9).

[31] David Louapre, Joel Niklaus, and Lewis Tunstall. "physics-intern: an autonomous agentic framework for physics research". In: *Hugging Face* (04/12/2026). URL: https://huggingface.co/spaces/huggingface/physics-intern (visited on 07/24/2026) (cit. on p. 4).

[32] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: https://arxiv.org/abs/2408.06292 (cit. on p. 4).

[33] Rahul Marchand, Art O Cathain, Jerome Wynne, Philippos Maximos Giavridis, Sam Deverett, John Wilkinson, Jason Gwartz, and Harry Coppock. *Quantifying Frontier LLM Capabilities for Container Sandbox Escape*. 2026. arXiv: 2603.02277 [cs.CR]. URL: https://arxiv.org/abs/2603.02277 (visited on 07/30/2026) (cit. on p. 9).

[34] Mojtaba omeili, Kurt Shuster, and Jason Weston. *Internet-Augmented Dialogue Generation*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland, 05/2022. DOI: 10.18653/v1/2022.acl-long.579. URL: https://aclanthology.org/2022.acl-long.579/ (visited on 07/28/2026) (cit. on p. 9).

[35] Stuart Russell and Peter Norvig. *Künstliche Intelligenz Ein moderner Ansatz*. Pearson Benelux B.V. (Zweigniederlassung Deutschland), 2023. ISBN: 9783868944303. URL: https://elibrary.pearson.de/book/99.150005/9783863263263 (visited on 07/26/2026) (cit. on p. 5).

[36] Erzhuo Shao, Yifang Wang, Yifan Qian, Zhenyu Pan, Han Liu, and Dashun Wang. "SciSciGPT: advancing human–AI collaboration in the science of science". In: *Nature Computational Science* (03/01/2026). URL: https://doi.org/10.1038/s43588-025-00906-6 (visited on 07/24/2026) (cit. on p. 4).

[37] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. *Announcing the Agent2Agent Protocol (A2A)*. 04/09/2025. URL: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/ (visited on 07/29/2026) (cit. on p. 10).

[38] InternAgent Team, Bo Zhang, Shiyang Feng, et al. *InternAgent: When Agent Becomes the Scientist – Building Closed-Loop System from Hypothesis to Verification*. 2025. arXiv: 2505.16938 [cs.AI]. URL: https://arxiv.org/abs/2505.16938 (cit. on p. 4).

[39] LCC) The Linux Foundation Project (LF Projects. *Model Context Protocol*. URL: https://github.com/modelcontextprotocol (visited on 07/29/2026) (cit. on p. 10).

[40] Romal Thoppilan, Daniel De Freitas, Jamie Hall, et al. *LaMDA: Language Models for Dialog Applications*. 2022. arXiv: 2201.08239 [cs.CL]. URL: https://arxiv.org/abs/2201.08239 (visited on 07/28/2026) (cit. on p. 9).

[41] Hanchen Wang, Tianfan Fu, Yuanqi Du, et al. "Scientific discovery in the age of artificial intelligence". In: *Nature* (08/01/2023). URL: https://doi.org/10.1038/s41586-023-06221-2 (visited on 07/24/2026) (cit. on p. 4).

[42] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. *SciBench: Evaluating College-Level Scientific Problem-*

Solving Abilities of Large Language Models. 2024. arXiv: 2307.10635 [cs.CL]. URL: https://arxiv.org/abs/2307.10635 (visited on 07/29/2026) (cit. on p. 11).

[43] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search*. 2025. arXiv: 2504.08066 [cs.AI]. URL: https://arxiv.org/abs/2504.08066 (cit. on p. 4).

[44] Wayne Xin Zhao, Kun Zhou, Junyi Li, et al. *A Survey of Large Language Models*. 2026. arXiv: 2303.18223 [cs.CL]. URL: https://arxiv.org/abs/2303.18223 (cit. on p. 5).

[45] Minhui Zhu, Minyang Tian, Xiaocheng Yang, et al. *Probing the Critical Point (CritPt) of AI Reasoning: a Frontier Physics Research Benchmark*. 2025. arXiv: 2509.26574 [cs.AI]. URL: https://arxiv.org/abs/2509.26574 (cit. on p. 11).

5 Appendix

6 Declaration of authorship

I hereby declare that I have written this thesis independently and without unauthorized assistance.

All sources and references used have been properly cited.

This thesis has not been submitted in the same or substantially similar form for any other degree or academic qualification.

Halle (Saale), 23rd August 2026

Janis Felix Jacobi

7 Acknowledgement

The authors gratefully acknowledge the computing time granted by the KISSKI project. The calculations for this research were conducted with computing resources under the project 9e28b773-a710-4a17-a242-523e92c812de.